

Databricks Connect: Distributed Machine Learning Walkthrough (Demo)

A Hands-On Guide to Distributed Machine Learning with Databricks Connect and Spark Connect

Authors: Alexander Booth

Introduction

This document provides a comprehensive walkthrough of leveraging Databricks Connect for distributed machine learning workflows. The demo illustrates the progression from local model training to distributed training using `pyspark.ml.connect`, highlighting current limitations and upcoming enhancements in Spark 4.0.

Environment Setup

Before diving into the notebooks, ensure the following prerequisites are met:

- **Databricks Connect Configuration:** Set up Databricks Connect on your local machine to establish a connection with a remote Databricks cluster. This enables you to run Spark code from your local environment, interacting seamlessly with the Databricks workspace. You will need the cluster ID and Databricks access token.
- **Cluster Requirements:**
 - **Databricks Runtime:** Version 16.x ML or higher.
 - **Access Mode:** Cluster access mode should be set to "Assigned" to allow proper permissions and resource allocation.
- **Python Environment:**
 - Ensure that your local Python environment has the necessary packages installed, including `databricks-connect`, `scikit-learn`, and `mlflow`.
- **MLflow Configuration:**
 - Ensure your MLflow tracking server has access configured in your workspace

With the environment configured, you're ready to explore the notebooks that demonstrate various aspects of distributed machine learning using Databricks Connect.

00 - Spark Connect Initialization

This notebook establishes a connection between your local development environment and a remote Databricks cluster using Databricks Connect.

Key Steps

1. Import Required Modules:

- a. Import the `DatabricksSession` from `databricks.connect`, along with a custom `config` module that contains your Databricks credentials and cluster information.

2. Set Environment Variables:

- a. Using values from the config module, configure environment variables for `DATABRICKS_HOST`, `DATABRICKS_TOKEN`, `DATABRICKS_CLUSTER_ID`, and `DATABRICKS_WORKSPACE_ID`.
- b. This setup ensures that the Databricks Connect client can authenticate and communicate with your Databricks workspace.

3. Initialize Spark Session:

- a. Create a Spark session using `DatabricksSession.builder.getOrCreate()`.
- b. This session allows you to execute Spark commands from your local environment, with computations running on the remote Databricks cluster.

4. Test the Connection:

- a. Run a simple Spark job to verify the connection:

```
df = spark.range(10).withColumnRenamed("id", "value")
df.show()
```
- b. This code creates a DataFrame with a range of numbers and displays it, confirming that the Spark session is operational.

5. Execute a SQL Query:

- a. Perform a basic SQL operation to further validate the setup:

```
spark.sql("SELECT COUNT(*) as even_count FROM range(100)
WHERE id % 2 = 0").show()
```

This query counts the number of even numbers in a range, demonstrating the ability to run SQL queries through the Spark session.

01 - Data Preparation for Machine Learning

This notebook focuses on preparing the dataset for machine learning tasks by loading it, processing it, and storing it in Unity Catalog for easy access.

Key Steps

1. Load the Dataset:

- a. Utilize `sklearn.datasets.load_breast_cancer` to load the breast cancer dataset as a pandas DataFrame. This dataset includes features and labels necessary for classification tasks.

2. Preprocess the Data:

- a. Combine the feature data and target labels into a single pandas DataFrame. For compatibility, rename columns to replace spaces with underscores.

3. Convert to Spark DataFrame:

- a. Transform the pandas DataFrame into a Spark DataFrame using `spark.createDataFrame(df_pd)`. This conversion enables distributed processing and integration with Spark's ML libraries.

4. Save to Unity Catalog:

- a. Write the Spark DataFrame to Unity Catalog using:

```
df.write.mode("overwrite").saveAsTable(f"{CATALOG_NAME}.{SCHEMA_NAME}.{TABLE_NAME}")
```

This step stores the data in a centralized location, making it accessible for subsequent notebooks and collaborative work.

02 - Local Model Training with MLflow Logging

This notebook demonstrates how to train a machine learning model locally using scikit-learn, while leveraging Databricks Connect to access data stored in Unity Catalog and log experiment details to MLflow hosted on Databricks.

Key Steps

1. Access Data via Databricks Connect:

- Utilize the established Spark session to read the prepared dataset from Unity Catalog into a pandas DataFrame for local processing.

2. Train a scikit-learn Model Locally:

- Split the dataset into training and testing subsets.
- Define a machine learning pipeline comprising preprocessing steps (e.g., `StandardScaler`) and a classifier (e.g., `LogisticRegression`).
- Set up a hyperparameter grid for tuning.
- Perform a grid search with cross-validation to identify the best model parameters.

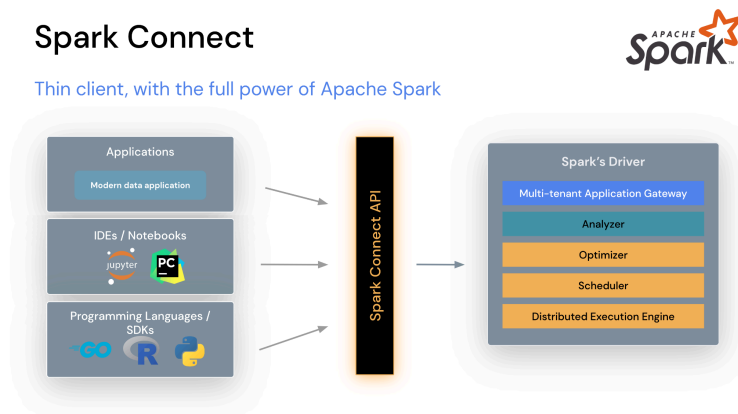
3. Log Experiment with MLflow:

- Initiate an MLflow run using `mlflow.start_run()`.
- Log model parameters, performance metrics (e.g., accuracy, ROC AUC), and the trained model itself to the MLflow tracking server on Databricks.
- Register the model in Unity Catalog for centralized management and future deployment.

These steps effectively train and track machine learning models locally while maintaining integration with Databricks' data and model management capabilities with MLflow.

03 - Attempted Distributed Training with `pyspark.ml`

This notebook explores the limitations encountered when attempting to perform distributed machine learning using the classic `pyspark.ml` API within a Spark Connect session.



Key Steps

1. Attempt to Define ML Pipeline:

- Construct a pipeline using components from the `pyspark.ml` library, such as `StandardScaler` and `LogisticRegression`.
- Set up a `CrossValidator` for hyperparameter tuning.

2. Encounter Runtime Errors:

- When executing the pipeline, errors are raised indicating the absence of a `SparkContext`.
- These errors stem from the fact that `pyspark.ml` relies on the JVM and a local `SparkContext`, which are not available in Spark Connect's client-server architecture.

3. Understand the Limitation:

- Spark Connect operates without an embedded JVM on the client side, making it incompatible with components that require direct JVM access.
- As a result, attempting to use `pyspark.ml` within a Spark Connect session leads to failures.

This notebook highlights the current limitations of using the classic `pyspark.ml` API with Spark Connect, setting the stage for exploring alternative approaches in subsequent notebooks.

To summarize, Databricks Connect uses a client-server model that runs Python code without an embedded JVM. As a result, components like `StandardScaler`, `Pipeline`, or `CrossValidator` from `pyspark.ml` cannot be used as they attempt to initialize Java objects that don't exist in the client environment.

In the next section, we'll use the newer `pyspark.ml.connect` API, which was designed specifically for Spark Connect to avoid these issues. It's not perfect, but it gets us much closer.

👉 [Learn more about distributed ML with Spark Connect](#)

04 - Distributed Model Training with `pyspark.ml.connect`

This notebook demonstrates how to perform distributed machine learning using the `pyspark.ml.connect` [module](#), which is designed to work seamlessly with Spark Connect and Databricks Connect.

Key Steps

1. Define the ML Pipeline:

- Utilize components from `pyspark.ml.connect`, such as `StandardScaler` for feature scaling and `LogisticRegression` for classification.
- Assemble these components into a `Pipeline` object.

2. Set Up Hyperparameter Tuning:

- Create a parameter grid using `ParamGridBuilder` to specify different values for hyperparameters like `maxIter`.
- Configure a `CrossValidator` to perform k-fold cross-validation, enabling robust model evaluation and selection.

3. Train the Model and Log with MLflow:

- Initiate an MLflow run using `mlflow.start_run()`.
- Fit the pipeline to the training data using the configured `CrossValidator`.
- Transform the test data to generate predictions.
- Evaluate model performance using metrics such as the area under the ROC curve (AUC) and accuracy.
- Log parameters, metrics, and the trained model to the MLflow tracking server hosted on Databricks.
- Register the model in Unity Catalog for centralized management and future deployment.

With these steps, you can effectively leverage `pyspark.ml.connect` to perform distributed machine learning tasks within a Spark Connect session, integrating seamlessly with Databricks' data and model management capabilities.

Conclusion & Looking Ahead to Spark 4.0

This demo has demonstrated how to leverage Databricks Connect for distributed machine learning workflows. We began with local model training using scikit-learn and MLflow for experiment tracking. Subsequently, we attempted to utilize the classic `pyspark.ml` API within a Spark Connect session, which resulted in failures due to the absence of a JVM in the client-server architecture.

To overcome this limitation, we employed the `pyspark.ml.connect` module, enabling successful distributed training and MLflow integration.

Looking ahead, Apache Spark 4.0 introduces significant enhancements that will further streamline distributed machine learning:

- **Unified Support for `pyspark.ml` in Spark Connect:** Spark 4.0 aims to provide native support for the classic `pyspark.ml` API within Spark Connect sessions, eliminating the need for the separate `pyspark.ml.connect` module. This development is tracked in [SPARK-49907](#) and is actively being developed, as seen in [PR #48791](#).
- **Enhanced Spark Connect Architecture:** The decoupled client-server model of Spark Connect allows for more flexible and scalable application development. Clients can interact with Spark clusters remotely using the DataFrame API, facilitating integration with various development environments and languages.
- **Operational Benefits:** With Spark Connect, applications can be developed and debugged interactively from IDEs, and dependencies can be managed independently on the client side. This architecture also improves stability and upgradability, as the Spark driver can be updated without affecting client applications.

By adopting these advancements in Spark 4.0, teams can expect a more seamless and efficient experience in developing and deploying distributed machine learning models.

References

- [Databricks Connect for Python](#)
- [Distributed ML with Spark Connect](#)
- [Spark Connect Overview for Spark 4.0](#)
- [SPARK-49907: Support spark.ml on Connect](#)
- [Apache Spark PR #48791](#)